

ICS 433 – Operating Systems

Distributed Resource Scheduler

Final Project Report

King Fahd University of Petroleum & Minerals

Rayan Alamri Mohammed Alsuhaibani Ali Almuzain Mohammed Yar

May 2026

Student Name	Student ID	Section	Main Contribution
Rayan Alamri	TODO: ID	TODO: Sec- tion	Master node, scheduler, queue, worker registry
Mohammed suhaibani	Al- TODO: ID	TODO: Sec- tion	Worker client, execu- tor, FFmpeg workloads
Ali Almuzain	TODO: ID	TODO: Sec- tion	Protocol, serialization, communication layer
Mohammed Yar	TODO: ID	TODO: Sec- tion	ncurses dashboard, UI integration

Note: Student ID and section values were not present in the repository metadata. They are marked as TODO so they can be filled before PDF submission.

Contents

1	Introduction	3
2	Problem Statement and Motivation	3
3	Complete Working Project	3
4	System Design	4
4.1	High-Level Architecture	4
4.2	Workflow Diagram	5
4.3	Module Explanation	6
5	Implementation Details	6
5.1	Network Protocol	6
5.2	Scheduling Algorithm	7
5.3	Fault Tolerance	7
5.4	Worker Execution and Process Isolation	7
5.5	Supported Workloads	8
5.6	FFmpeg Video Pipeline	8
5.7	Tools and Technologies	8
6	Results and Testing	9
6.1	Test Cases	9
6.2	Output Screenshots	9
6.3	Performance	11
7	Challenges and Solutions	11
7.1	TCP Partial Reads	11
7.2	Thread Safety and Lock Ordering	12
7.3	Worker Disconnect Race	12
7.4	Heartbeat Timeout	12
7.5	FFmpeg Segment Cleanup	12
7.6	Dashboard Integration	12
8	Future Improvements	12
9	Team Contribution	13
10	Code Submission and README	13
10.1	Requirements	13
10.2	Native Run Commands	13
10.3	Docker Fixed-Core Simulation	13
11	Conclusion	14

1 Introduction

The Distributed Resource Scheduler is a mini operating-system style project implemented in C. It demonstrates how a master node can coordinate a pool of worker nodes across a local network, assign jobs, receive results, monitor worker health, and recover from failures. The project focuses on operating-system concepts rather than high-level frameworks: sockets, threads, process creation, pipes, synchronization, scheduling, worker liveness, and resource-aware execution.

The final version supports multiple workload types, including numerical workloads, parallel matrix chunks, Mandelbrot row chunks, Monte Carlo sampling, and distributed FFmpeg video processing. It also includes a terminal dashboard, Docker-based simulation, task requeueing, heartbeat timeout detection, and performance measurement for one-worker versus three-worker FFmpeg execution.

2 Problem Statement and Motivation

Many computers in a lab or local network are underutilized. A single machine can become slow when processing CPU-intensive or media workloads, while other machines remain idle. The goal of this project is to build a small distributed scheduler that can:

- accept jobs from a user or dashboard,
- divide selected workloads into independent tasks,
- assign tasks to available workers,
- collect and aggregate results,
- detect worker failure and requeue incomplete work,
- display progress and worker status in real time.

The motivation is similar to systems such as Hadoop, Kubernetes, or cluster job runners, but implemented at a smaller scale using POSIX primitives. The project therefore provides practical experience with OS-level abstractions while producing a working distributed system.

3 Complete Working Project

The final project is a complete working mini distributed scheduler. It includes:

- a master server that accepts workers and submit clients over TCP,
- a worker process that registers with the master and executes assigned tasks,
- a scheduler with least-loaded selection and round-robin tie breaking,
- a thread-safe task queue and worker registry,
- fixed-size network messages with byte-order conversion,
- fork-based process isolation for workloads,
- a live ncurses dashboard,
- FFmpeg video splitting, segment processing, merging, and segment cleanup,
- heartbeat monitoring with a 15-second stale-worker timeout,
- automatic requeueing of unfinished tasks from disconnected or stale workers,
- Docker and Docker Compose support for simulation.

4 System Design

4.1 High-Level Architecture

The system follows a master-worker architecture. The master owns global state: task queue, worker registry, scheduler, job summary, and dashboard. Workers are stateless compute nodes from the scheduler's point of view; they receive a task, execute it, and return a result.

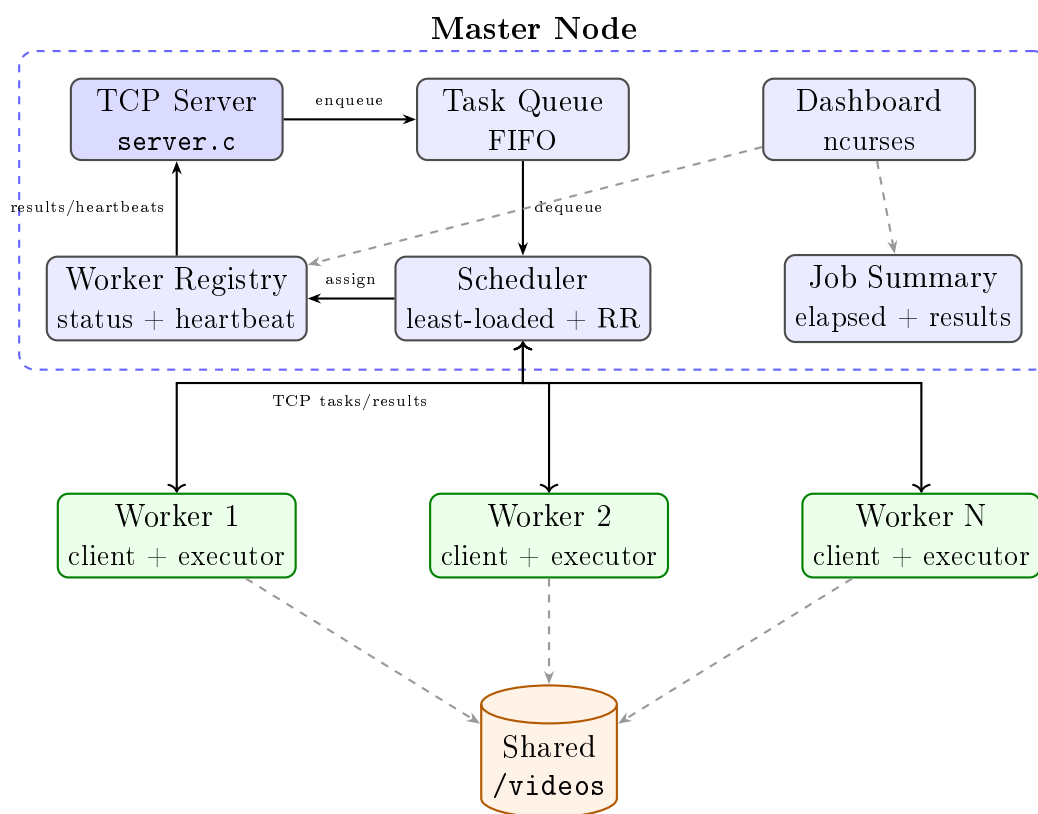


Figure 1: Framework and architecture diagram for the Distributed Resource Scheduler.

4.2 Workflow Diagram

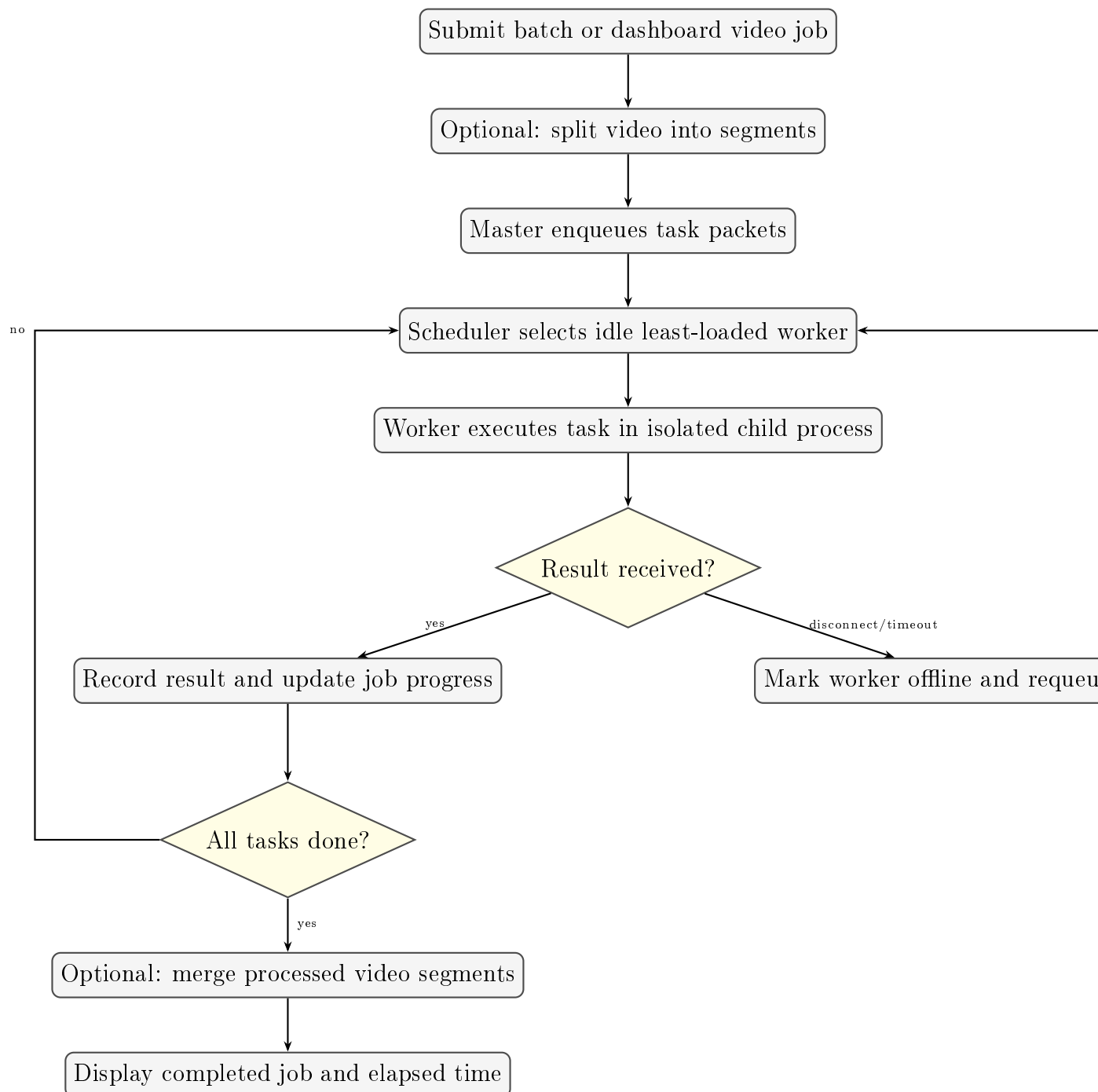


Figure 2: End-to-end task workflow, including fault recovery.

4.3 Module Explanation

Module	Files	Responsibility
Master server	src/master/server.c	TCP accept loop, worker threads, job summaries, dashboard submission API, video pipeline helpers
Scheduler and registry	src/master/scheduler.c/.h	Worker table, scheduling algorithm, heartbeat timeout, offline marking
Task queue	src/master/queue_mgr.c/.h	Thread-safe FIFO queue and requeue operations
Status API	src/master/status.c/.h	Snapshot generation for dashboard and JSON reporting
Worker client	src/worker/client.c/.h	Connects to master, sends heartbeats, receives tasks, sends results
Executor	src/worker/executor.c/.h	Forks child processes and runs workloads in isolation
Protocol	src/shared/protocol.c/.h, models.h	Fixed-size message frame, byte-order conversion, reliable send/receive
Dashboard	src/ui/dashboard.c/.h	ncurses monitoring UI and job submission forms
Scripts and Docker	Makefile, Dockerfile, docker-compose.yml, scripts/	Build, simulation, video split/submit/merge helpers

5 Implementation Details

5.1 Network Protocol

All master-worker and submit-master communication uses a fixed-size `NetworkPayload`. Each field is converted to network byte order before transmission and restored to host byte order after reception. The protocol avoids partial-read bugs by using `send_full()` and `recv_full()`, which loop until the full payload has been transferred.

Listing 1: Network payload shape.

```

1 typedef struct {
2     uint32_t type;
3     uint32_t worker_id;
4     uint32_t task_id;
5     uint32_t command_code;
6     uint32_t argument;
7     uint32_t result;
8 } NetworkPayload;

```

5.2 Scheduling Algorithm

The final scheduler uses a least-loaded strategy with round-robin tie breaking:

1. dequeue a task from the FIFO queue,
2. scan for idle workers,
3. choose the idle worker with the fewest completed tasks,
4. if several workers are tied, use a rotating round-robin index,
5. mark the worker busy before sending the task,
6. on send failure, requeue the task and mark the worker offline.

This balances work better than pure round-robin when workers are heterogeneous. A faster worker can complete more tasks and return to idle sooner, while the tie breaker prevents the same worker from always being selected first.

5.3 Fault Tolerance

Fault tolerance is implemented using three mechanisms:

- **Socket disconnect handling:** if a worker socket closes while it is busy, the master requeues the worker's unfinished task.
- **Send failure handling:** if the scheduler cannot send a task to a selected worker, the task is returned to the queue.
- **Heartbeat timeout:** if no heartbeat arrives for more than 15 seconds, the scheduler marks the worker offline and requeues its current task if one exists.

The project intentionally treats disconnected tasks as incomplete rather than failed. A task is marked failed only when a worker returns a workload-specific failure result, such as an FFmpeg timeout or missing segment.

5.4 Worker Execution and Process Isolation

Each worker executes tasks through `executor_run_task()`. The executor forks a child process, runs the workload inside that child, writes the result through a pipe, and exits. The parent reads the pipe, waits for the child with `waitpid()`, and sends the result to the master. This isolates workload crashes from the worker's network state.

5.5 Supported Workloads

Cmd	Name	Description
1	prime	Count primes up to an upper bound.
2	matrix	Compute checksum of one full virtual matrix.
3	prime_range	Parallel prime counting over independent ranges.
4	monte_carlo	Parallel Monte Carlo sampling for estimating pi.
5	mandelbrot	Parallel row-chunk checksum for Mandelbrot rows.
6	ffmpeg_segment	Parallel FFmpeg processing of video segments.
7	ffmpeg_script	Parallel custom FFmpeg script execution per segment.
8	matrix_parallel	Parallel matrix checksum using 100-row chunks.

5.6 FFmpeg Video Pipeline

For video workloads, the master or helper scripts split an input video into files such as `part_000.mp4`, `part_001.mp4`, and so on. Each segment becomes a separate task. Workers process segments into `videos/processed/`, and the master merges processed segments into `videos/final/`. After a segment is successfully processed, its original input segment is deleted because it is no longer needed.

5.7 Tools and Technologies

- C language with POSIX APIs
- TCP sockets
- pthreads
- fork, pipe, waitpid, exec-style process management
- ncurses dashboard
- FFmpeg and ffprobe
- Docker and Docker Compose
- GNU Make
- LaTeX for documentation

6 Results and Testing

6.1 Test Cases

Test	Purpose
Queue FIFO test	Verifies tasks leave the queue in submission order.
Queue requeue test	Verifies orphaned tasks can be placed back into the queue.
Registry tests	Verifies worker add, idle selection, busy exclusion, and offline marking.
Protocol roundtrip	Verifies network byte-order conversion preserves all fields.
Prime executor test	Verifies prime workload correctness.
Matrix executor test	Verifies full matrix checksum correctness.
Matrix parallel test	Verifies matrix chunk checksum matches the full matrix result for a small matrix.
Monte Carlo test	Verifies sampling result is within an acceptable range.
Mandelbrot test	Verifies deterministic row-chunk checksum behavior.
FFmpeg integration	Verifies segment processing, merge, output file generation, and segment cleanup.
Fault tolerance	Verifies task requeue when a worker disconnects or times out.

6.2 Output Screenshots

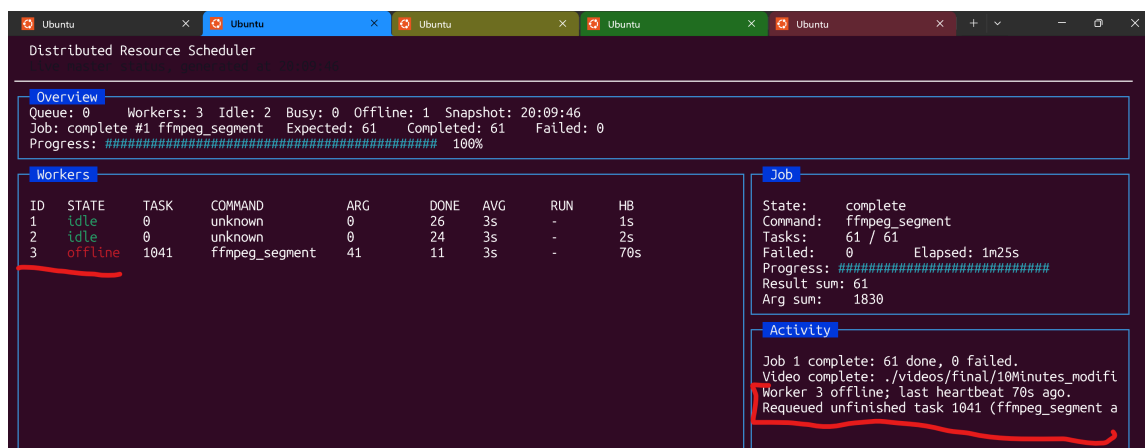


Figure 3: Fault tolerance test: a worker is offline and the dashboard reports the unfinished task that was requeued.

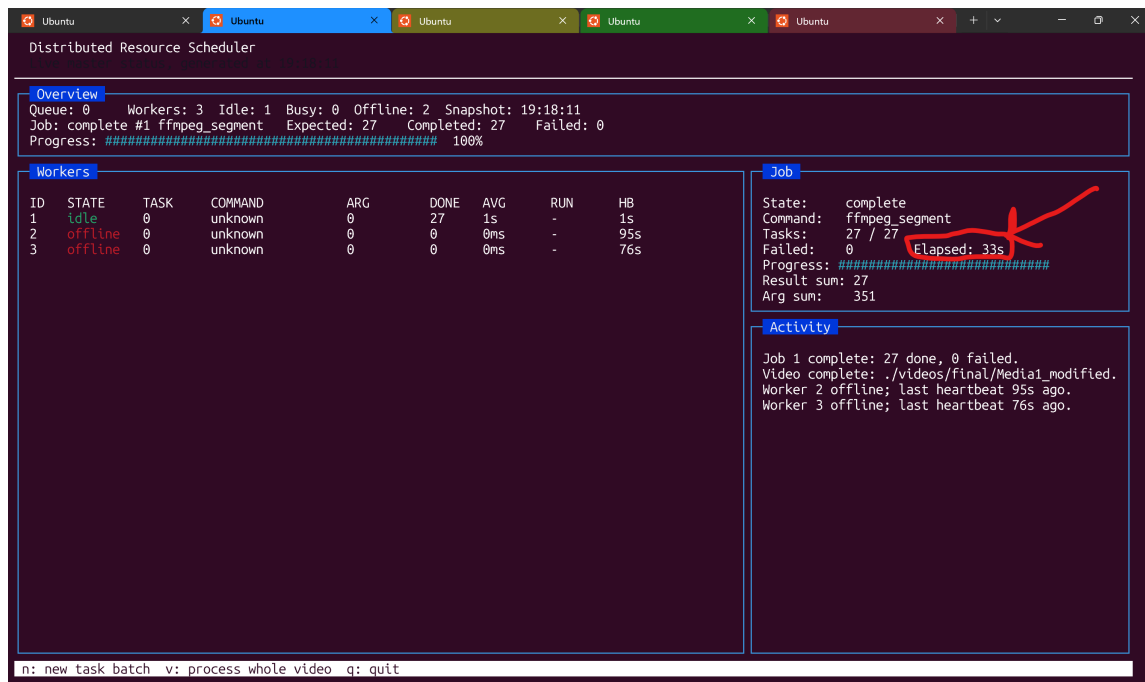


Figure 4: FFmpeg performance run using one active worker. The dashboard shows the completed job elapsed time.

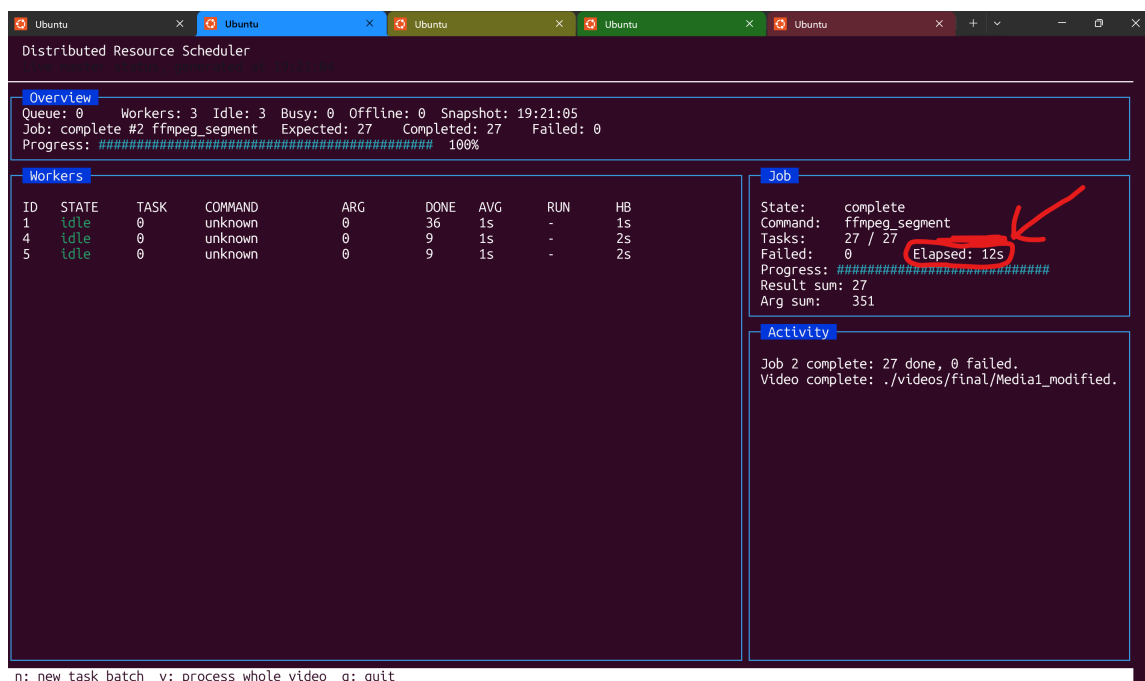


Figure 5: FFmpeg performance run using three active workers. The elapsed time is lower due to parallel segment processing.

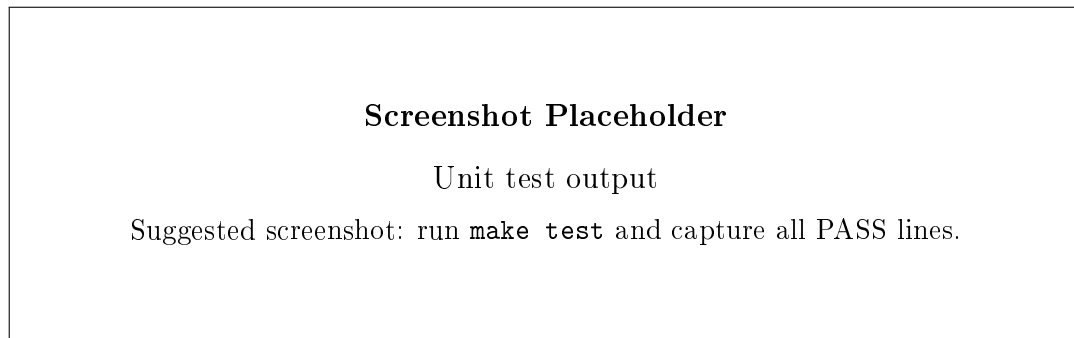


Figure 6: Placeholder: Unit test output

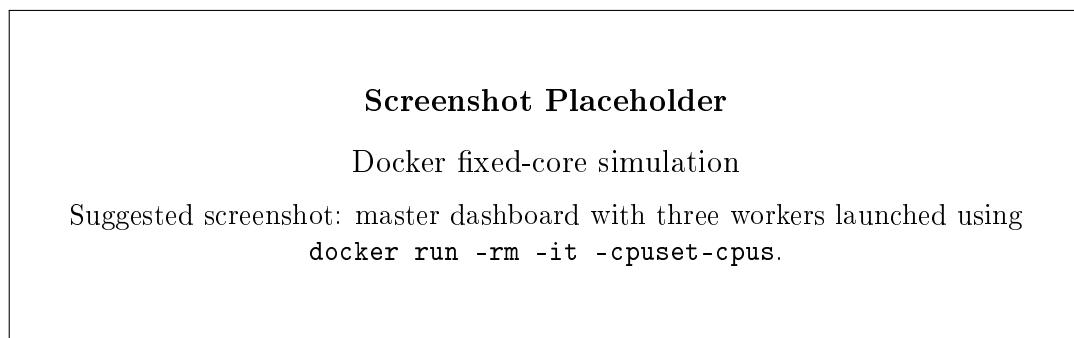


Figure 7: Placeholder: Docker fixed-core simulation

6.3 Performance

The FFmpeg segment workload demonstrates the benefit of parallel execution. In the one-worker screenshot, the dashboard reports an elapsed time of 33 seconds for a 27-segment job. In the three-worker screenshot, the dashboard reports 12 seconds for the same 27-segment workload.

Configuration	Workers	Elapsed Time	Speedup
Single-worker FFmpeg	1	33 s	1.00x
Three-worker FFmpeg	3	12 s	2.75x

The speedup is not exactly 3.00x because of scheduling overhead, worker startup timing, FFmpeg internal threading, video segment size variation, and shared disk I/O. However, the improvement shows that the scheduler successfully distributes independent segment tasks across workers.

7 Challenges and Solutions

7.1 TCP Partial Reads

TCP is a byte stream, so a single `recv()` call is not guaranteed to return a full struct. The solution was to implement `recv_full()` and `send_full()` so every message frame is transmitted completely.

7.2 Thread Safety and Lock Ordering

The master has several shared structures, including the queue, registry, and job summary. To avoid deadlock, the implementation avoids holding registry and queue locks at the same time. When a task must be requeued, the task is copied while holding the registry lock, then the lock is released before queue insertion.

7.3 Worker Disconnect Race

When a worker disconnects while busy, its current task can be lost if the worker state is cleared too early. The solution was to snapshot the assigned task before marking the worker offline and to keep the last unfinished task visible for dashboard activity reporting.

7.4 Heartbeat Timeout

Socket disconnect is not always immediate in real networks. The final implementation adds a scheduler-side timeout: if a worker has not sent a heartbeat for more than 15 seconds, it is marked offline and its unfinished task is requeued.

7.5 FFmpeg Segment Cleanup

Processed video segments are needed for merging, but original input segments are not needed after successful processing. The master now deletes each original input segment after receiving a successful FFmpeg result, reducing disk usage during long runs.

7.6 Dashboard Integration

The dashboard needed to read live state without blocking scheduler or network threads. The solution was to expose a snapshot API that copies the queue, worker, and job state under locks, then renders the copied state in ncurses.

8 Future Improvements

- Add persistent job history so completed job summaries remain visible after new submissions.
- Store performance measurements in CSV for graph generation.
- Add a configurable heartbeat timeout through environment variables.
- Improve FFmpeg task metadata so the dashboard can display video names and segment counts per job.
- Add real Raspberry Pi deployment scripts using NFS or Samba shared storage.
- Add stronger retry limits so a repeatedly failing task does not loop forever.
- Add a web dashboard or JSON API client for easier remote monitoring.

9 Team Contribution

Member	Contribution
Rayan Alamri	Master server, queue manager, worker registry, scheduler, job summary integration, fault tolerance behavior.
Mohammed suhaibani	Worker client, heartbeat thread, executor, task isolation, FFmpeg worker operations.
Ali Almuzain	Shared protocol, message model, byte-order conversion, reliable socket send/receive helpers.
Mohammed Yar	Terminal dashboard design, activity panel, job panel, worker table, video workflow UI.

10 Code Submission and README

The final code is organized under the `src/` directory and built through the root `Makefile`. The project also includes a detailed `README.md` and `MANUAL.md` with requirements, build steps, native execution, Docker execution, FFmpeg pipeline steps, and fault tolerance testing.

10.1 Requirements

- Linux, WSL, macOS, or another POSIX-like environment
- `gcc`
- `make`
- `pthread` support
- `ncurses` development package
- `ffmpeg` and `ffprobe` for video workloads
- Docker and Docker Compose for container simulation

10.2 Native Run Commands

```
make
./bin/master
./bin/worker 127.0.0.1 9090
./bin/submit -h 127.0.0.1 -p 9090 -c 8 -n 10 -a 0 -s 100 -e 1000
```

10.3 Docker Fixed-Core Simulation

```
docker build -t drs .
docker network create drs-net

docker run --rm -it --name drs-master \
  --network drs-net \
  -p 9090:9090 \
  -e MASTER_PORT=9090 \
  -e MASTER_DASHBOARD=1 \
```

```
-e TERM=xterm-256color \  
-v "$PWD/videos:/videos" \  
-v "$PWD/logs:/logs" \  
drs master
```

Each worker is then launched in a separate terminal:

```
docker run --rm -it --name drs-worker-1 \  
--network drs-net \  
--cpuset-cpus="0,1" \  
-e VIDEO_DIR=/videos \  
-v "$PWD/videos:/videos" \  
-v "$PWD/logs:/logs" \  
drs worker drs-master 9090
```

11 Conclusion

The Distributed Resource Scheduler successfully implements a working mini OS-style distributed execution platform. It combines OS-level concurrency, socket communication, scheduling, worker monitoring, fault recovery, and real media-processing workloads. The final system demonstrates that independent jobs can be distributed across workers, that failures can be recovered through requeueing, and that performance improves when parallel workloads such as FFmpeg segment processing are executed by multiple workers.